Separation Logic I

The heap, the separating conjunction and the frame rule

Cristina David

Lecture 10

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

Where we are

Two lectures of Hoare logic. It reasons about variables.

But variables are independent boxes: assigning to x cannot possibly change y. That is what made the assignment axiom a *substitution*.

Where we are

Two lectures of Hoare logic. It reasons about variables.

But variables are independent boxes: assigning to x cannot possibly change y. That is what made the assignment axiom a *substitution*.

Today we add the heap — and that assumption stops holding.

Why the assignment axiom breaks

Add pointers to the IMP language: $[x]$ is "the heap cell that x points to".

Apply the old axiom to the mutation $[x] := 4$ with postcondition $Q \equiv ([y] = 3)$:

$$\{Q[4/[x]]\} \;\; [x] := 4 \;\; \{Q\}$$

Why the assignment axiom breaks

Add pointers to the IMP language: $[x]$ is "the heap cell that x points to".

Apply the old axiom to the mutation $[x] := 4$ with postcondition $Q \equiv ([y] = 3)$:

$$\{Q[4/[x]]\}\ \ [x] := 4\ \ \{Q\} \quad \rightsquigarrow \quad \{[y] = 3\}\ \ [x] := 4\ \ \{[y] = 3\}$$

Does it look fine?

Why the assignment axiom breaks

Add pointers to the IMP language: $[x]$ is "the heap cell that x points to".

Apply the old axiom to the mutation $[x] := 4$ with postcondition $Q \equiv ([y] = 3)$:

$$\{Q[4/[x]]\}\ \ [x] := 4\ \ \{Q\} \quad \rightsquigarrow \quad \{[y] = 3\}\ \ [x] := 4\ \ \{[y] = 3\}$$

Does it look fine?



before: $[y] = 3$ ✓

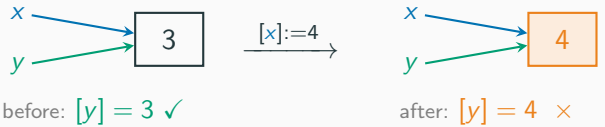
Why the assignment axiom breaks

Add pointers to the IMP language: $[x]$ is "the heap cell that x points to".

Apply the old axiom to the mutation $[x] := 4$ with postcondition $Q \equiv ([y] = 3)$:

$$\{Q[4/[x]]\}\ \ [x] := 4\ \ \{Q\} \quad \rightsquigarrow \quad \{[y] = 3\}\ \ [x] := 4\ \ \{[y] = 3\}$$

Does it look fine?



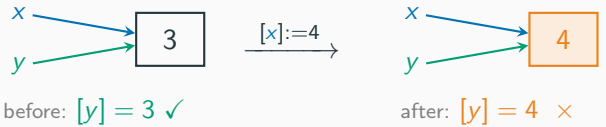
Why the assignment axiom breaks

Add pointers to the IMP language: $[x]$ is "the heap cell that x points to".

Apply the old axiom to the mutation $[x] := 4$ with postcondition $Q \equiv ([y] = 3)$:

$$\{Q[4/[x]]\}\ \ [x] := 4\ \ \{Q\} \quad \rightsquigarrow \quad \{[y] = 3\}\ \ [x] := 4\ \ \{[y] = 3\}$$

Does it look fine?



The problem: aliasing

x and y hold the *same* address, so $[x] := 4$ also changes $[y]$. Substitution is syntactic: it cannot see that two *different names* denote *one cell*.

The model

States, now with a heap

A state is now two components:

$$\text{store } s : \mathrm{Var} \to \mathrm{Val} \qquad\qquad \text{heap } h : \mathrm{Loc} \rightharpoonup \mathrm{Val}$$

- The store is the old σ: variables to values. Total.
- The heap maps *locations* to values, and is partial — its domain $\mathrm{dom}(h)$ is the finite set of currently-allocated cells.

States, now with a heap

A state is now two components:

$$\text{store } s : \mathrm{Var} \to \mathrm{Val} \qquad\qquad \text{heap } h : \mathrm{Loc} \rightharpoonup \mathrm{Val}$$

- The store is the old σ: variables to values. Total.
- The heap maps *locations* to values, and is partial — its domain $\mathrm{dom}(h)$ is the finite set of currently-allocated cells.

We write $s, h \vDash P$. Assertions will now describe the shape of the heap, not just the values of variables.

Heap assertions

The empty heap

$s, h \vDash \mathsf{emp} \quad \text{iff} \quad \mathrm{dom}(h) = \varnothing.$

Points-to: a single cell

$s, h \vDash x \mapsto v \quad \text{iff} \quad \mathrm{dom}(h) = \{\, s(x) \,\} \quad \text{and} \quad h(s(x)) = s(v).$

Heap assertions

The empty heap

$s, h \vDash \mathsf{emp} \quad \text{iff} \quad \mathrm{dom}(h) = \varnothing.$

Points-to: a single cell

$s, h \vDash x \mapsto v \quad \text{iff} \quad \mathrm{dom}(h) = \{\, s(x) \,\} \quad \text{and} \quad h(s(x)) = s(v).$

Read $x \mapsto v$ as: "the heap is exactly one cell, at address x, holding v".

Small axioms of separation logic

Commands for the heap

Four new commands, each with a small axiom — stated on the smallest heap that makes sense, its *footprint*.

mutation	$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$	
lookup	$\{x \mapsto v\}\ \ y := [x]\ \ \{y = v \wedge x \mapsto v\}$	(y fresh)
allocation	$\{\mathsf{emp}\}\ \ x := \mathsf{new}(a)\ \ \{x \mapsto a\}$	
disposal	$\{x \mapsto _\}\ \ \mathsf{dispose}(x)\ \ \{\mathsf{emp}\}$	

What if we have a larger heap?

The small axioms only cover a *one-cell* heap. What if we have three cells and we update *one*?



What if we have a larger heap?

The small axioms only cover a *one-cell* heap. What if we have three cells and we update *one*?



$$\{\, x \mapsto 3 \land y \mapsto 3 \land z \mapsto 7 \,\} \quad [x] := 4 \quad \{\, x \mapsto 4 \land y \mapsto 3 \land z \mapsto 7 \,\}^{\dagger}$$

[†]Here we relax $\mapsto$: $x \mapsto v$ means the heap *contains* a cell at x holding v, not that it is the *whole* heap.

What if we have a larger heap?

The small axioms only cover a *one-cell* heap. What if we have three cells and we update *one*?



$$\{\, x \mapsto 3 \land y \mapsto 3 \land z \mapsto 7 \,\} \;\; [x] := 4 \;\; \{\, x \mapsto 4 \land y \mapsto 3 \land z \mapsto 7 \,\}^{\dagger}$$

We need a way to say "these cells are separate":

$$\{\, x \mapsto 3 \land y \mapsto 3 \land z \mapsto 7 \land x \neq y \land x \neq z \,\} \;\; [x] := 4 \;\; \{\, x \mapsto 4 \land y \mapsto 3 \land z \mapsto 7 \,\}$$

[†]Here we relax $\mapsto$: $x \mapsto v$ means the heap *contains* a cell at x holding v, not that it is the *whole* heap.

The real question

This is global reasoning, where we keep the *whole* heap in mind.
But the command only touches *one* cell.

The real question

This is global reasoning, where we keep the *whole* heap in mind.
But the command only touches *one* cell.

The goal: local reasoning

We want to reason about the cells a command *touches*, and ignore the rest of the heap — without proving, every time, that "the rest" really is untouched.

The separating conjunction of separation logic

The separating conjunction

$P * Q$ — "P and Q, on disjoint heaps"

$s, h \vDash P * Q$ iff h splits as $h = h_1 \uplus h_2$ with

$$h_1 \perp h_2, \qquad s, h_1 \vDash P, \qquad s, h_2 \vDash Q.$$

($\uplus$ is union of heaps with disjoint domains.)

The separating conjunction

$P * Q$ — "P and Q, on disjoint heaps"

$s, h \vDash P * Q$ iff h splits as $h = h_1 \uplus h_2$ with

$$h_1 \perp h_2, \qquad s, h_1 \vDash P, \qquad s, h_2 \vDash Q.$$

($\uplus$ is union of heaps with disjoint domains.)

$P * Q$ asserts P and Q hold on separate parts of the heap. The split is an *existential*: "there is some way to divide the heap so that. . . ".

The frame rule

The frame rule

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

The frame rule

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

In words: if c transforms P into Q using only the footprint P, then it leaves any disjoint heap R untouched.

The frame rule

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

In words: if c transforms P into Q using only the footprint P, then it leaves any disjoint heap R untouched.

This is the heart of separation logic.

The frame rule at work

The small axiom for mutation:

$$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$$

The frame rule at work

The small axiom for mutation:

$$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$$

Frame on an arbitrary disjoint rest-of-heap R:

$$\{(x \mapsto _) * R\}\ \ [x] := a\ \ \{(x \mapsto a) * R\}$$

The frame rule at work

The small axiom for mutation:

$$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$$

Frame on an arbitrary disjoint rest-of-heap R:

$$\{(x \mapsto _) * R\}\ \ [x] := a\ \ \{(x \mapsto a) * R\}$$

R might be a million other cells. We never re-verify them.

The side condition is not optional

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

Let's drop it.

Take $\{\mathsf{emp}\}\ \ x := \mathsf{new}(4)\ \ \{x \mapsto 4\}$, where $\mathrm{mod}(c) = \{x\}$.

The side condition is not optional

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

Let's drop it.

Take $\{\mathsf{emp}\}\ \ x := \mathsf{new}(4)\ \ \{x \mapsto 4\}$, where $\mathrm{mod}(c) = \{x\}$.

Frame with $R \equiv x \mapsto 3$, which mentions x:

$$\{x \mapsto 3\}\ \ x := \mathsf{new}(4)\ \ \{x \mapsto 4 * x \mapsto 3\}$$

The side condition is not optional

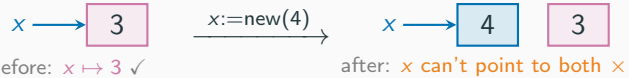
$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

Let's drop it.

Take $\{\mathsf{emp}\}\ \ x := \mathsf{new}(4)\ \ \{x \mapsto 4\}$, where $\mathrm{mod}(c) = \{x\}$.

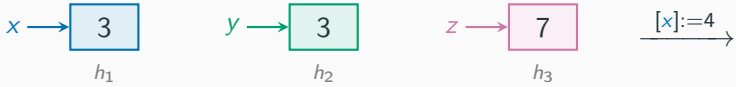
Frame with $R \equiv x \mapsto 3$, which mentions x:

$$\{x \mapsto 3\}\ \ x := \mathsf{new}(4)\ \ \{x \mapsto 4 * x \mapsto 3\}$$



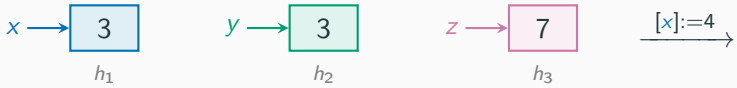
The larger heap, revisited

The three-cell example from before — now with $*$ and the frame rule:



The larger heap, revisited

The three-cell example from before — now with $*$ and the frame rule:

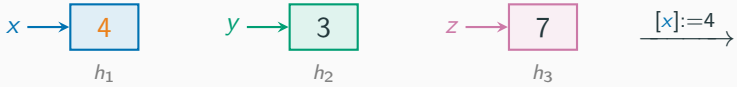


$$\frac{\{x \mapsto 3\}\ \ [x] := 4\ \ \{x \mapsto 4\}}{\{x \mapsto 3 * \underbrace{y \mapsto 3 * z \mapsto 7}_{R}\}\ \ [x] := 4\ \ \{x \mapsto 4 * \underbrace{y \mapsto 3 * z \mapsto 7}_{R}\}}\ \text{Frame}$$

Premise: the small axiom, on h_1 alone. Side condition: $[x] := 4$ modifies no variable in R✓

The larger heap, revisited

The three-cell example from before — now with $*$ and the frame rule:



$$\frac{\{x \mapsto 3\}\ \ [x] := 4\ \ \{x \mapsto 4\}}{\{x \mapsto 3 * \underbrace{y \mapsto 3 * z \mapsto 7}_{R}\}\ \ [x] := 4\ \ \{x \mapsto 4 * \underbrace{y \mapsto 3 * z \mapsto 7}_{R}\}}\ \text{Frame}$$

Premise: the small axiom, on h_1 alone. Side condition: $[x] := 4$ modifies no variable in R✓

Wrap

Today

- State = store + partial heap. Assertions describe heap shape: emp, $x \mapsto v$.
- The separating conjunction $P * Q$ splits the heap into disjoint parts. $* \neq \wedge$: $*$ *builds in disjointness*.
- Small axioms on footprints + the frame rule = local, modular reasoning.

Next time

We can describe a single cell. Real programs use data structures (lists, trees) which are unboundedly many cells.

We need recursive assertions: $\mathsf{list}(x)$, $\mathsf{lseg}(x, y)$.

Next time

We can describe a single cell. Real programs use data structures (lists, trees) which are unboundedly many cells.

We need recursive assertions: $\mathsf{list}(x)$, $\mathsf{lseg}(x, y)$.

Separation Logic II (Lecture 11): inductive predicates.

Questions?

ATiPL 2026 Minute Sheet